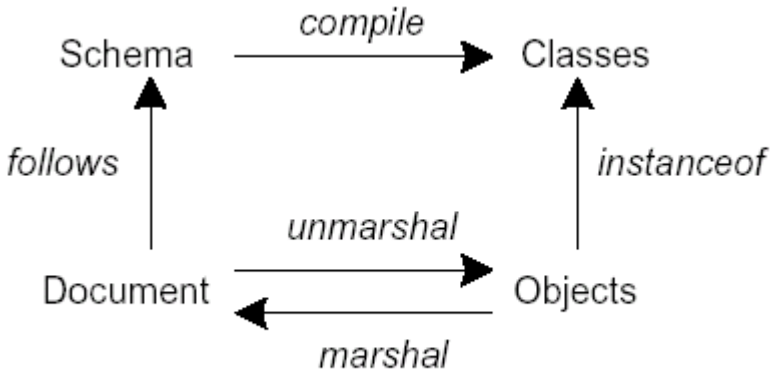
1. Principes - JAXB

JAXB (**J**ava **A**pi for **X**ml **B**inding) est une api assez récente permettant d'établir une **association (correspondance)** entre :

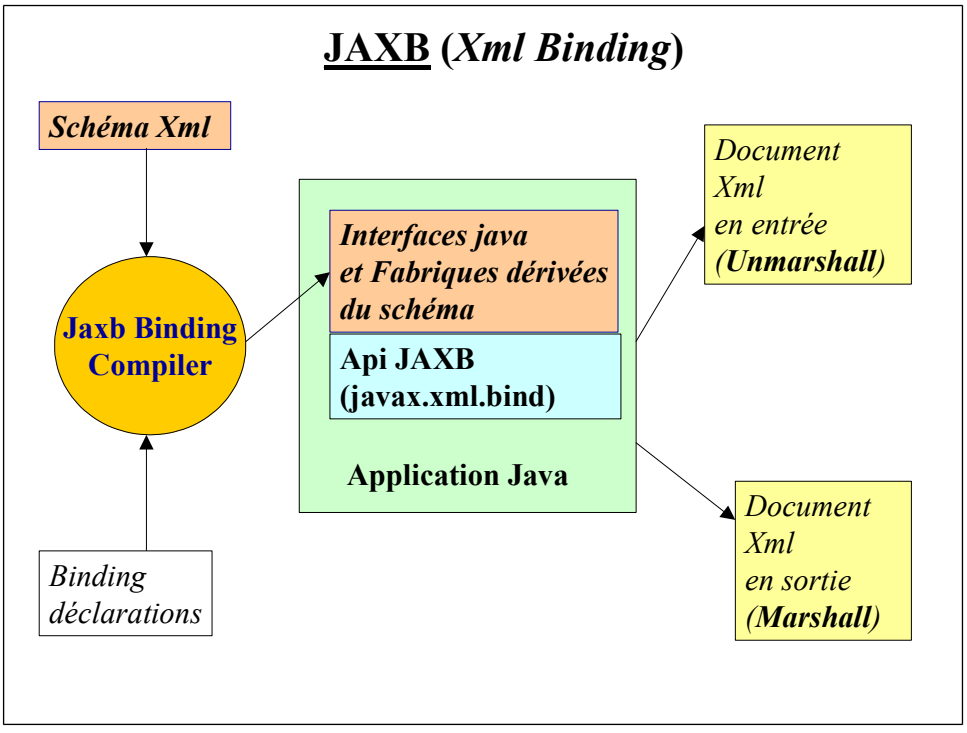
un **document XML** (dont le *type* est décrit par un *schéma XML*)

et

un **ensemble de classes java** (formant ensemble une structure arborescente) .



Une fois que les classes java ont été générées conformément au schéma , on peut alors effectuer très rapidement une "**sérialisation XML**" entre des instances des classes java (comportant certaines valeurs) et un fichier XML (instance du schéma) .



2. Versions 1 et 2 (JAXB)

La première version de JAXB peut fonctionner avec le jdk 1.4 car elle n'utilise pas d'annotations Java5 au niveau de son paramétrage.

La nouvelle version JAXB2 est quant à elle spécifique à Java 5 (et 6) .

La suite de ce chapitre sera illustré avec la version JAXB2.

3. Mise en oeuvre (JAXB2)

Fixer quelques variables d'environnement:

set JAVA_HOME=C:\Prog\JAVA_5\jdk1.5.0_09

```
set JAXB_HOME=C:\Prog\Sun\jwsdp-2.0\jaxb
set JAXB_LIBS=%JAXB_HOME%\lib


set CLASSPATH=%CLASSPATH%;%JAXB_LIBS%\jaxb-api.jar;
              %JAXB_LIBS%\jaxb-xjc.jar;%JAXB_LIBS%\jaxb-impl.jar;
```

Invocation du compilateur de binding xml (générateur de code java en fonction d'un schéma xml):

```
%JAXB_HOME%\bin\xjc.bat -p pack1 po.xsd
```

```
  Usage: xjc [-options ...] <schema>
  Options:
    -nv       :  do not perform strict validation of the input schema(s)
    -d <dir>  :  generated files will go into this directory
    -p <pkg>  :  specifies the target package
    -readOnly :  generated files will be in read-only mode
    -help     :  display this help message
```

NB: ***xjc.bat*** lance en interne la commande java suivante:

```
java -jar %JAXB_HOME%/lib/jaxb-xjc.jar
```

4. Exemple de binding (JAXB2)

Schéma XML (biblio.xsd):

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<xsd:element name="bibliographie" type="BibliographieType"/>
<xsd:element name="comment" type="xsd:string"/>
<xsd:complexType name="BibliographieType">
  <xsd:sequence>
    <xsd:element name="titre" type="xsd:string"/>
    <xsd:element name="sujet" type="xsd:string"/>
    <xsd:element ref="comment" minOccurs="0"/>
    <xsd:element name="livre" type="Livre"  minOccurs="1"
maxOccurs="unbounded" />
  </xsd:sequence>
  <xsd:attribute name="date" type="xsd:date"/>
</xsd:complexType>
```

```
<xsd:complexType name="Livre">
        <xsd:sequence>
         <xsd:element name="titre" type="xsd:string"/>
         <xsd:element name="auteur" type="xsd:string"/>
         <xsd:element name="editeur" type="xsd:string"/>
        </xsd:sequence>
</xsd:complexType>
</xsd:schema>
```

Document Xml (biblio.xml = instance possible du schéma)

```
<?xml version="1.0"?>
<bibliographie date="01/01/2002">
 <titre> Bibliographie sur Programmation Xml en java </titre>
 <sujet> Livres evoquant la programmation Xml en langage Java </sujet>
 <livre>
     <titre>XML, langage et applications</titre>
     <auteur>Alain Michard (INRIA)</auteur>
     <editeur>Eyrolles</editeur>
 </livre>
 <livre>
     <titre>Construire une application XML ...</titre>
     <auteur>J.C. Bernadac , F.Knab </auteur>
     <editeur>Eyrolles</editeur>
 </livre>
</bibliographie>
```

%JAXB_HOME%\bin\xjc.bat -p mybiblio biblio.xsd

==> Interfaces java et fabriques générées:

```
parsing a schema...
compiling a schema...
mybiblio\BibliographieType.java
mybiblio\Livre.java
mybiblio\ObjectFactory.java
```

```
....
import javax.xml.bind.annotation.AccessType;
import javax.xml.bind.annotation.XmlAccessorType;
import javax.xml.bind.annotation.XmlAttribute;
import javax.xml.bind.annotation.XmlType;
...
```

BibliographieType.java

```
@XmlAccessorType(AccessType.FIELD)
@XmlType(name = "BibliographieType", propOrder = {
   "titre",
   "sujet",
   "comment",
   "livre"
})
public class BibliographieType {

   protected String titre;
   protected String sujet;
   protected String comment;
```

```
    protected List<Livre> livre;
    @XmlAttribute
    protected XMLGregorianCalendar date;
public String getTitre() {        return titre;
    }

    public void setTitre(String value) {        this.titre = value;
    }

    public String getSujet() {        return sujet;
    }

    public void setSujet(String value) {        this.sujet = value;
    }

    public String getComment() {        return comment;
    }

    public void setComment(String value) {        this.comment = value;
    }

    public List<Livre> getLivre() {
        if (livre == null) {
            livre = new ArrayList<Livre>();
        }
        return this.livre;
    }
    public XMLGregorianCalendar getDate() {        return date;
    }

    public void setDate(XMLGregorianCalendar value) {        this.date = value;
    }

}
```

Livre.java

```
@XmlAccessorType(AccessType.FIELD)
@XmlType(name = "Livre", propOrder = {
    "titre",
    "auteur",
    "editeur"
})
public class Livre {

    protected String titre;
    protected String auteur;
    protected String editeur;

   // + public get/set
}
```

ObjectFactory.java

```
package mybiblio;

import javax.xml.bind.JAXBElement;
import javax.xml.bind.annotation.XmlElementDecl;
import javax.xml.bind.annotation.XmlRegistry;
import javax.xml.namespace.QName;
import mybiblio.BibliographieType;
import mybiblio.Livre;
import mybiblio.ObjectFactory;

@XmlRegistry
public class ObjectFactory {

    private final static QName _Comment_QNAME = new QName("", "comment");
    private final static QName _Bibliographie_QNAME =
                            new QName("", "bibliographie");


    public ObjectFactory() {
    }

    public Livre createLivre() {
        return new Livre();
    }

    public BibliographieType createBibliographieType() {
        return new BibliographieType();
    }

    @XmlElementDecl(namespace = "", name = "comment")
    public JAXBElement<String> createComment(String value) {
        return new JAXBElement<String>(_Comment_QNAME,
                                String.class, null, value);
    }

    @XmlElementDecl(namespace = "", name = "bibliographie")
    public JAXBElement<BibliographieType>
                createBibliographie(BibliographieType value) {
        return new JAXBElement<BibliographieType>(_Bibliographie_QNAME,
                                        BibliographieType.class,
                                        null, value);
    }

}
```

5. Structure de l'api JAXB

Le principal package de **JAXB** est **javax.xml.bind**

La classe abstraite **JAXBContext** est le point d'entrée de l'API.

```
JAXBContext   jctx =
   JAXBContext.newInstance( "com.mycompany.pack1:com.mycompany.pack2" );
```

Le paramètre d'entrée de la méthode statique **newInstance** correspond à la liste de(s) package(s) de classes et d'interfaces java dérivées du schéma xml.

Une fois instancié, le contexte JAXB peut servir à créer les éléments suivants:

```
Unmarshaller um = jctx.createUnmarshaller(); // pour remonter des objets en mémoire
                                             // suite à la lecture d'un flux xml
Marshaller m = jctx.createMarshaller(); // pour générer un flux xml (fichier)
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);
```

Dé-sérialisation d'un flux xml:

```
JAXBElement<XxxType> xElement =
            (JAXBElement<XxxType>)  um.unmarshal( new File( "xxx.xml" ) );
XxxType xxxObj = (XxxType) xElement.getValue();
```

Sérialisation vers un flux xml:

```
JAXBElement<YyyType> yyyElement = objectFactory.createYyyy(yyyObj);
m.marshal( yyyElement, myOutputStream );
```

Création d'une nouvelle instance via la fabrique générée au sein du package (fruit de la compilation du schéma Xml):

```
com.mycompany.pack1.ObjectFactory objectFactory
                        = new com.mycompany.pack1.ObjectFactory();
com.mycompany.pack1.Xxx po =  objectFactory.createXxx();
```

6. Exemple de code (JAXB2)

```
package myprog;

import javax.xml.bind.*; // JAXB
import javax.xml.datatype.*;
import mybiblio.*; // Result of XML Binding
import java.io.*;



public class MyJaxbApp {
```

```
   private static XMLGregorianCalendar getDateAsXmlGreogorianCalendar() {
            try {
                return DatatypeFactory.newInstance()
                        .newXMLGregorianCalendar(new GregorianCalendar());
            } catch (DatatypeConfigurationException e) {
                throw new Error(e);
            }
        }

      public static void main(String[] args) {

      try {
      JAXBElement<BibliographieType> biblioElement = null;
      JAXBContext jctx = JAXBContext.newInstance("mybiblio");
      Unmarshaller um = jctx.createUnmarshaller();
      Marshaller m = jctx.createMarshaller();
      m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);


      biblioElement=(JAXBElement<BibliographieType>)
                 um.unmarshal( new File( "biblio.xml" ) );
      BibliographieType biblioObj = (BibliographieType)biblioElement.getValue();
      // ....
      // create an element for marshalling
      biblioElement = objectFactory.createBibliographie(biblioObj);
      m.marshal( biblioElement, System.out );

      ObjectFactory objectFactory = new ObjectFactory();
      BibliographieType biblioObj3 = objectFactory.createBibliographieType();
      biblioObj3.setDate( getDateAsXmlGreogorianCalendar());
      biblioObj3.setTitre("titre biblio");
      biblioObj3.setSujet("sujet qui va bien");
      biblioObj3.setComment("mon commentaire qui ne sert a rien");

      Livre l1 = objectFactory.createLivre();
      l1.setAuteur("Auteur qui ecrit bien");
      l1.setEditeur("Eyrolles");
      l1.setTitre("Titre du livre");

      Livre l2 = objectFactory.createLivre();
      l2.setAuteur("Auteur fou");
      l2.setEditeur("Masson");
      l2.setTitre("Titre2 du livre2");

      biblioObj3.getLivre().add(l1);
      biblioObj3.getLivre().add(l2);

      /*  Validator ==> in JAXB1 but no more in JAXB2
            Validator v= jctx.createValidator();
            v.validate(biblioObj3);
            */

      // create an element for marshalling
       biblioElement = objectFactory.createBibliographie(biblioObj3);
      // marshalling
      m.marshal( biblioElement, new FileOutputStream("biblio3.xml") );

      }
      catch(Exception ex) { ex.printStackTrace(); }

      }
}
```

7. Utilisation de JAXB2 sur une classe java (sans xsd)

La simple annotation `@XmlRootElement` (de *javax.xml.bind.annotation*) permet à JAXB2 d'utiliser simplement une classe de donnée

```
@XmlRootElement()//defaultName="notification"
public class Notification {
	private String subject; //+get/set
	private String message; //+get/set

	@Override
	public String toString() {
		return "Notification [subject=" + subject + ", message=" + message
				+ "]";
	}
	// +get/set
}
```

Les quelques lignes suivantes suffisent à transformer une chaîne de caractère xml en un objet java :

```
...
try {
JAXBContext context = JAXBContext.newInstance(Notification.class);
Unmarshaller um = context.createUnmarshaller();
Notification notObj =  (Notification)
		um.unmarshal(new StreamSource(new StringReader(xmlString)));
System.out.println(notObj.toString());

} catch (JAXBException e) {
	e.printStackTrace();
}
```

à utiliser avec

```
xmlString="<notification><subject>sujetXY</subject>"
			+"<message>messageAB</message></notification>";
```